Exploring NPC Behaviors in Games through Finite Automata

Hardik Gupta
School of Computing.
Amrita School of Engineering.
Bengaluru, India
hardikgupta3011@gmail.com

Ananya Ganapathi
School of Computing.
Amrita School of Engineering.
Bengaluru, India
ananyaganapathi.27@gmail.com

Parikshith Sivakumar
School of Computing.
Amrita School of Engineering.
Bengaluru, India
parikshith.sr19@gmail.com

Aditya Elango
School of Computing.
Amrita School of Engineering.
Bengaluru, India
adityaelango2004@gmail.com

***Abstract*—A non-playing character is one which performs an important role in the world building of all the games present. The role of non-playing character is not common to all the games. A non-playing character's role may vary from an enemy who attacks the main character in games such as Call of Duty and Assasin's Creed to an ally who helps the main character reach some goal in games such as God of War and Uncharted. These roles can be explored and simulated through the use of a finite automata.**

***Index Terms*—Behavior modeling, Gaming, State diagram, NPC, Simulation, Finite automata, Transition table, Transition diagram.**

I. Introduction

Non-playing character in any gaming environment is the one which is not in the control of the main character but it functions based on what the main character is performing. Non-playing characters may take on roles of enemies, allies, traders, helpers, chasers or neutral to the players.

Depending on these roles the non-playing character's behavior is decided. Suppose if the non-playing character is an enemy then when the player approaches them, they will either attack or run from the player otherwise they remain idle. While the non-playing character is attacking, that time also they may have to decide about which weapon to use or what defense method to use.

Similarly non-playing characters can act as an ally also by helping the player in their quests or when the player needs resources. All this can be determined using different states of a finite automata.

Modeling these different behaviors should be done in a systematic manner for ease of implementation of these characters in the game.

In this paper, these different behaviors of a non-playing character can be understood and presented in different games through the use of finite automata. Finite automata provides a structured and organised way of working on these behaviors of non-playing characters. It will help to develop a better understanding of the different roles taken by non-playing characters in a better manner.

II. Literature Survey

- The study [1] talks about the advancement of game development and focuses on the increasing demand for advanced AI engines and graphic engines. It recommends the use of Finite State Machines for creating intelligent NPCs, it introduces the Hierarchical FSM concept for NPC behaviour management. The paper highlights the benefits of using FSM in creating different action patterns for the NPC. This idea can be implemented in our project to create a practical way in which NPC characters can act during a game.
- The paper [2] introduces the use of Goal-Oriented Action Planning in 3D hide-and-seek games, it focuses on using effective hiding strategies by NPCs. The study shows that NPCs using goal oriented action planning with limited hiding spots show higher aggression and success rates, complicating seeker detection. The paper tells us to integrate goal oriented action planning into NPC behaviour to enhance gameplay challenges and improve player experience. This research can be used in our project to understand the mechanics of goal oriented action planning implementation and its impact on gameplay dynamics.
- The research paper [3] shows the challenges we face in creating smart Non-Playable Characters in computer games. Hierarchical Abstract Machines (HAMs) could be used as an alternative to traditional finite state machines. HAMs are flexible and efficient; they can change NPC behaviour by decomposing tasks into manageable subtasks. We can implement HAMs in our project to enhance players' experience. HAMs is a wonderful solution to enhance NPC behaviour in games. It can also be used to control NPC activities in a game.
- The study [4] promotes the addition of educational content into simulation games, to improve user engagement. By using Finite State Machines in a Survival simulation

game to generate random gameplay based on events to challenge players on a remote island. The game's aim is to teach survival skills and produce variability in player experiences in unfamiliar contexts. The paper promotes the idea of Finite State Machines to enhance player experience in computer games. This paper helps us to make our project more efficient by creating random gameplay.

- The paper [5] talks about the importance of animation control in action games and then it compares the Finite State Machine and Hierarchical Finite State Machine to manage character animations. It promotes HFSM's benefits, such as clustering and hierarchy, to streamline computational processes and enhance maintainability. The paper integrates HFSM into Unity's animator interface, demonstrating superior performance through ten experiments. The paper provides valuable insights into HFSM's effectiveness and sets the stage for future research in this domain.
- The paper [6] discusses how to use a Non-Deterministic Finite Automata (NDFA) to model player behavior in Pro Evolution Soccer game. The model converts game controller inputs into symbols and the player's state into finite states. All states can move freely and perform actions like sprinting or placing the ball. The model suggests easier transitions between the closely linked states. Also, wrong inputs at specific moments can lead to fouls, like trying to cross without any possession. The model also identified closely linked states, suggesting easier transitions between them. Finally, the NDFA successfully captured potential gameplay sequences based on user inputs.
- This research [7] explores utilizing automata theory, particularly DFA and NDFA, to design a Ludo game for multiplayer gaming. Using Alan Turing's foundational work, the paper highlights the important role of computation and automata in game design, emphasizing simplicity and efficiency. The proposed Ludo game illustrates bug-free performance and high accuracy, which promises to enhance multiplayer experiences on many devices. The paper talks about Future directions which included employing advanced automata, such as Turing machines and push-down automata, to address real-time bugs and optimize game-play with limited resources, therefore enhancing the multiplayer game design.
- The study [8] explains the use of automata in designing of a game, focusing on the development of Hangaroo, a variation of the Hangman game. The study is based on finite automata, specifically deterministic and non-deterministic, due to their simplicity and effectiveness. Developers can enhance their game design by including automata in game theory. The paper presents a detailed explanation of Hangaroo's design, explaining how automata theory checks the game's flow and logic.

III. METHODOLOGY

A. WORKFLOW

a) Introduction: The study focuses on the designing of Non- playing Character using non-deterministic finite automata to analysis the behavior of different NPCs across different games. The main motivation for developing an NPC Non-deterministic Finite automaton is to focus on the complexities of NPC behavior modeling in video games. So, by using the finite automata, we are going to design a formal and systematic method to represent NPC behavior, allowing for more understanding of the NPCs states at different stages of the game.

b) Objective: The objective of the project is to develop and implement a Finite Automaton-based framework for modeling Non-Playing Character (NPC) behavior in video games. We aim to achieve the following goals using NFA:

1. We will be providing a systematic framework for tailoring diverse NPC behaviors, including traders, chasers, allies and enemies found in video games.

2.The NPCs will be exhibiting adaptive nature behavior based on the player's actions, environmental factors, and game state.

3.Analysis the states of the NPCs at various stages of the game. It helps us in better understanding of the how the NPCs states change during the gameplay.

c) Literature review: The literature surveys show rec ent game development trends, with a focus on how we are making use of automata theory to improve non-playing character behavior and gameplay. It promotes using Finite State Machines, Goal-Oriented Action Planning, and Hierarchical Abstract Machines for better NPC management and immersive gaming experiences. Also, many multiplayer game designs and player interactions are based on using Non-Deterministic Finite Automata (NDFA).

d) Structure: The study focuses on creating an NPC behaviors model using finite automata. The chaser NPC system manages chasing behavior, whereas the trader NPC algorithm handles interactions during any sort of resource trading scenarios. The ally NPC will respond to player movement and commands. Finally, the enemy NPC includes searching, chasing, and attacking the player.With the help of this method, we will be able to understand better about the NPCs role in gameplay.

B. ALGORITHM

a) Algorithm for Chaser NPC: 1. NPC is in idle state.

2. When the player starts game, NPC transitions to Chase1 state.

3. If the main player is hit by an obstacle while in Chase1 state,

then NPC transitions to Chase2 state.

Else, NPC remains in Chase1 state.

4. If the main player is hit by an obstacle while NPC is in Chase2 state,

then NPC transitions to Caught state (final state).

Else, if the main player escapes, then NPC transitions back to Chase1 state.

b) Algorithm for Trader NPC: 1. NPC is in idle state.

2. When the main character enters, NPC transitions to greet state.

3. When the main character requests resources, NPC transitions to search state.

4. If the requested item is found,
then NPC transitions to sell state.
If one more request is made, return to step 3.
Else, NPC transitions to sorry state.
If one more request is made, return to step 3.

5. If the player exits
then NPC transitions to idle state

c) Algorithm for Ally NPC: 1. NPC is in idle state.

2. If the player moves, NPC transitions to follow state.

3. If the player is attacked by an enemy, NPC transitions to defend state.

4. If the player commands the NPC to attack,
then NPC transitions to attack state.
If NPC wins, then NPC transitions back to idle state.
Else, NPC transitions to dead state.

5. If the player commands the NPC to support,
then NPC transitions to help state.

d) Algorithm for Enemy NPC: 1. NPC is in idle state.

2. If the NPC is searching for the player, it transitions to run state.

3. If the NPC detects the player, it transitions to chase state.

4. If the NPC catches the player, it transitions to attack state.

5. If the NPC wins,
then transition to idle end state (final state).
Else, if the player escapes, transition back to chase state.
If the NPC is defeated, transition to dead state (final state).

C. DESIGN

Using the algorithm given in the previous section the following design is produced using the JFLAP application.

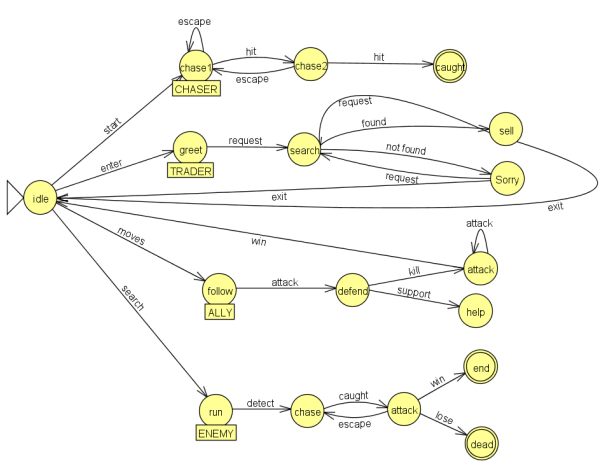

Fig. 1. NPC behavior design

D. PROCESS EXPLANATION

This paper gives an overview of few of the different roles performed by a non-playing character. This is explained with the help of the JFLAP application which helps in simulation of the finite automata and many other types of automata. For now, this paper has discussed four main roles which are: chaser, trader, ally and enemy.

a) Chaser: A chaser can be observed in many famous games such as Subway Surfers and Temple Run where the NPC chases the main character. In the Fig. 1 it can be observed that when the game has not started the NPC has no role so it remains in the idle state but when the player starts the game it starts chasing the main player. Here chase1 refers to the off-screen chasing while the chase2 refers to on-screen chasing which occurs as a warning when player hits an obstacle. If the player doesn't hit again and escapes then state changes to chase1 again. But when this happens twice the player is caught and the game ends reaching the final state.

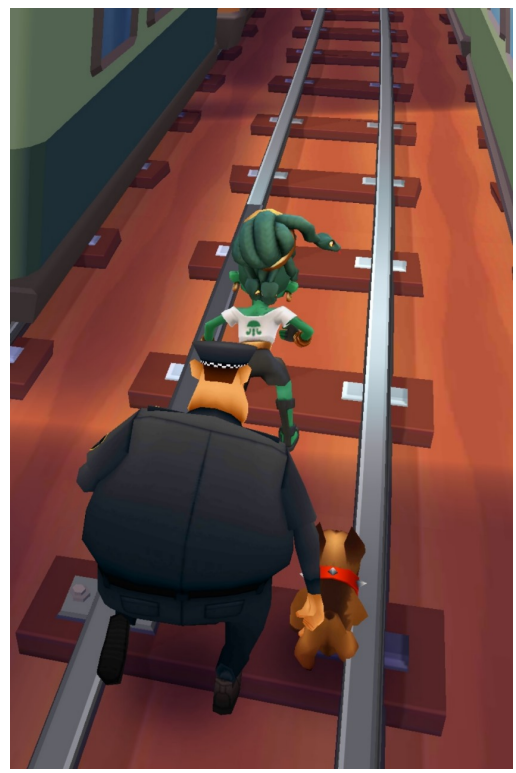

Fig. 2. Example of chaser NPC

b) Trader: Trader is someone whose role is to provide and exchange resources from the player. Generally, when a player approaches the trader NPC, a greeting message is given by the trader which is defined by the greet state of Fig. 1. Then when a request for resource is provided then the NPC transitions to the search state where it looks for the asked resource. If found it will sell or will display a sorry message. But if the player wants to ask for multiple resources after the first request the state will return to the search state this will keep on repeating hence this doesn't have a final state. The Fig.

3 taken from Albion online, depicts a trader who sells various goods and weapons to mercenaries,knights and healers.

Fig. 3. Example of trader NPC

c) Ally: Another role of NPC is an ally who supports and helps the main character for their various quests and challenges. Like all other roles NPC is in idle state until no action is taken place by the main character to change its state. When ally sees the player moves it transitions to the follow state. If an enemy attacks the ally they transition to the defend state to protect the player. If the player orders the ally to kill the enemy they transition to attack and if NPC is ordered to support the player then they transition to help state where they may heal or powerup the player.

Fig. 4. Example of Ally NPC

d) Enemy: Enemy NPCs are the most fundamental part for an adventurous gaming experience, providing challenges, obstacles and missions for the player to overcome. Some of the examples include the Super Mario bro and GTA Vice city III. These characters are associated mostly with aggressive behaviors, where it searches for the player and after finding it tries to fight with the player. Generally, the enemy NPC will be in the idle state until it notices a player nearby. If the player has been detected and tries to run away, then NPC will start running behind to get hold of the player. Before the player escapes, the NPC must begin attacking. Now after it starts attacking there are chances for the player to escape, in such scenario the state back traces again to chase. Otherwise, it goes to the final state: win or lose.

Fig. 5. Example of Enemy NPC

IV. RESULT

The result of the simulation of the chaser and enemy can be observed where the input string of ”start escape hit escape hit escape hit hit ” is provided as input where the output can be observed in Fig. 6.

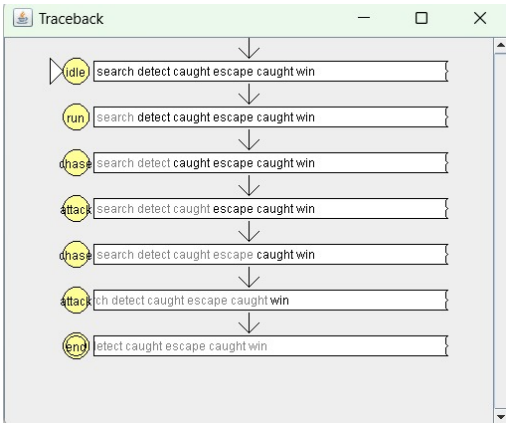

Fig. 6. Enemy simulation

Similarly, for the chaser simulation the input string of ”search detect caught escape caught win ” is given. Fig. 7 shows the result of the simulation of enemy NPC.

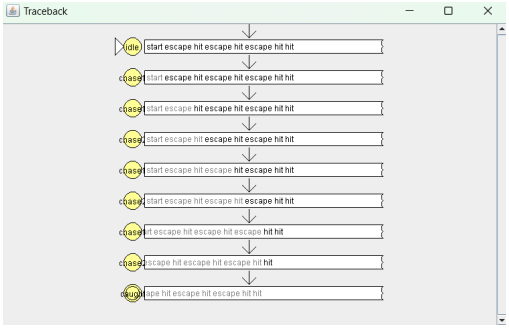

Fig. 7. Chaser simulation

V. CONCLUSION

- The paper demonstrated how we have implemented NFAs, and it also talks about the versatility of Finite Automata (NFAs) in modeling the Non-Playing Characters behaviors (NPCs) within video games. With the help of

NFAs, we have established a structure which represents the diverse NPC characters, like traders, chasers, allies, and enemies. This framework offers a very structured and efficient approach to defining NPC behavior patterns, enabling developers to create dynamic and responsive characters that enhance the overall player experience.

Moreover, by applying NFAs for NPC behavior modeling offers a systematic methodology that enhances the clarity and organization of game development projects. Using the NFAs, developers can systematically go through the different states and transitions that occur in terms of the NPC behaviors, providing a very easy and automatic approach to game design. This approach not only helps us to understand the complicated NPCs interactions but it also benefits the developers by providing an efficient development process, thus enabling them to alter the gaming experience completely which will be also enhance the player experience .Therefore, integrating NFAs into NPC modeling represents a advancement in game design methodology, helping out the developers to create more engaging and interactive gaming experiences while maintaining a better understanding on their workflows.

REFERENCES

[1] Kwon, K.D. and Park, K.R., 2006, October. Behavior control of intelligent multi NPCs using vickrey auction system and hierarchical finite state machine. In 2006 SICE-ICASE International Joint Conference (pp. 3821-3826). IEEE.

[2] Suyikno, D.A. and Setiawan, A., 2019, October. Feasible npc hiding behaviour using goal oriented action planning in case of hide-and-seek 3d game simulation. In 2019 Fourth International Conference on Informatics and Computing (ICIC) (pp. 1-6). IEEE.

[3] Xiaoqin, D., Qinghua, L. and Jianjun, H., 2009, August. Applying hierarchical reinforcement learning to computer games. In 2009 IEEE International Conference on Automation and Logistics (pp. 929-932). IEEE.

[4] Kuswadayan, I., Herumurti, D., Hariadi, R.R., Wildianurahman, M., Yunanto, A.A. and Arifiani, S., 2019, July. Survival education for user on unknown islands using simulation games. In 2019 12th International Conference on Information and Communication Technology and System (ICTS) (pp. 307-311). IEEE.

[5] Dewi, E.N.F., Rachman, A.N. and Hartadji, R.H.Z., 2023, December. Implementation of Hierarchical Finite State Machine for Controlling 2D Character Animation in Action Video Game. In 2023 Eighth International Conference on Informatics and Computing (ICIC) (pp. 1-6). IEEE.

[6] Scholar, A., 2020. Application of Non-Deterministic Finite Automata to Pro Evolution Soccer (PES) Game. 1Abubakar Muhammad Shafii; 2Manko Abubakar Abdulrahman; 3Ibrahim Aliyu; 4Odo, Chika Emelda.

[7] Ali, K.F., Kalyan, V. and Kumar, K.A., 2019, March. Design and implementation of ludo game using automata theory. In 2019 Innovations in Power and Advanced Computing Technologies (i-PACT) (Vol. 1, pp. 1-6). IEEE.

[8] Yusoff, Y., Nazmi, A., Izzat, M., Irwan, M.S., Zulfahmi, M. and Sallehuddin, R., 2021. Hangman–Hangaroo Game Design Using Automata Theory. International Journal of Innovative Computing, 11(1), pp.7-11.

[9] Hopcroft JE, Motwani R, Ullman JD. Introduction to Automata Theory, Languages and Computation. Third Edition, Pearson; 2006.

[10] Sipser M. Introduction to the Theory of Computation. Third Edition Cengage Publishers; 2005.

[11] Martin JC. Introduction to Languages and the Theory of Computation. Fourth Edition McGraw-Hill; 2010.

[12] Wang, D. and Chen, Y., 2021, June. Visual teaching design of compile principle based on JFLAP. In 2021 2nd International Conference on Artificial Intelligence and Education (ICAIE) (pp. 366-370). IEEE.